Arquitectura AWS y Flujo de Ejecución

Diagrama de referencia de la infraestructura y del recorrido de un documento, de principio a fin. Para la explicación conceptual (qué hace el sistema, sin código) ver `COMO_FUNCIONA.md`; para el recorrido con referencias a archivo:línea, ver `FLUJO_EJECUCION.md`.

> Versión PDF de este documento (mismos diagramas, para imprimir o compartir sin depender de un visor de Markdown): `ARQUITECTURA.pdf`.

Modelo

El pipeline usa **Amazon Bedrock** con el perfil de inferencia cross-region `us.anthropic.claude-haiku-4-5-20251001-v1:0` (**Claude Haiku 4.5**) tanto para clasificar como para extraer campos — el modelo desplegado hoy en dev y prod, verificado contra AWS Cost Explorer (ver `COSTOS.md`). Configurable sin tocar código vía `.env.dev` / `.env.production`:

Variable	Para qué	Default
`BEDROCK_MODEL_ID`	Modelo del flujo estándar (clasificación + extracción single-shot)	`us.anthropic.claude-haiku-4-5-20251001-v1:0`
`BEDROCK_EXTRACTOR_MODEL_ID`	Modelo del pipeline de expediente (segmentación + extracción por documento)	igual a `BEDROCK_MODEL_ID` — se puede usar uno más fuerte solo ahí

Cambiar el modelo requiere **redeploy** del ambiente correspondiente (la variable se hornea en la Lambda al desplegar).

Arquitectura AWS

Todo el stack es serverless (`DocumentValidationStack-Dev` en dev, `DocumentValidationStack` en prod — dos stacks aislados y completos, ver `AMBIENTES_Y_SEGURIDAD.md`). La Step Function la crea el blueprint `@cdklabs/cdk-appmod-catalog-blueprints`; este repo aporta las tres Lambdas propias (Upload, Enrichment, Query) y toda la configuración del catálogo.

Los colores de los nodos siguen la paleta oficial por categoría de servicio de AWS (la misma de los AWS Architecture Icons), para que cada pieza se identifique de un vistazo:

🟠 Compute (Lambda) · 🟢 Storage (S3) · 🔵 Database (DynamoDB) · 🩷 App Integration (SQS, EventBridge, Step Functions) · 🟣 Networking (API Gateway) · 🔴 Security (KMS) · 🩵 Machine Learning (Bedrock, Textract) · ⚫ Cliente / externo

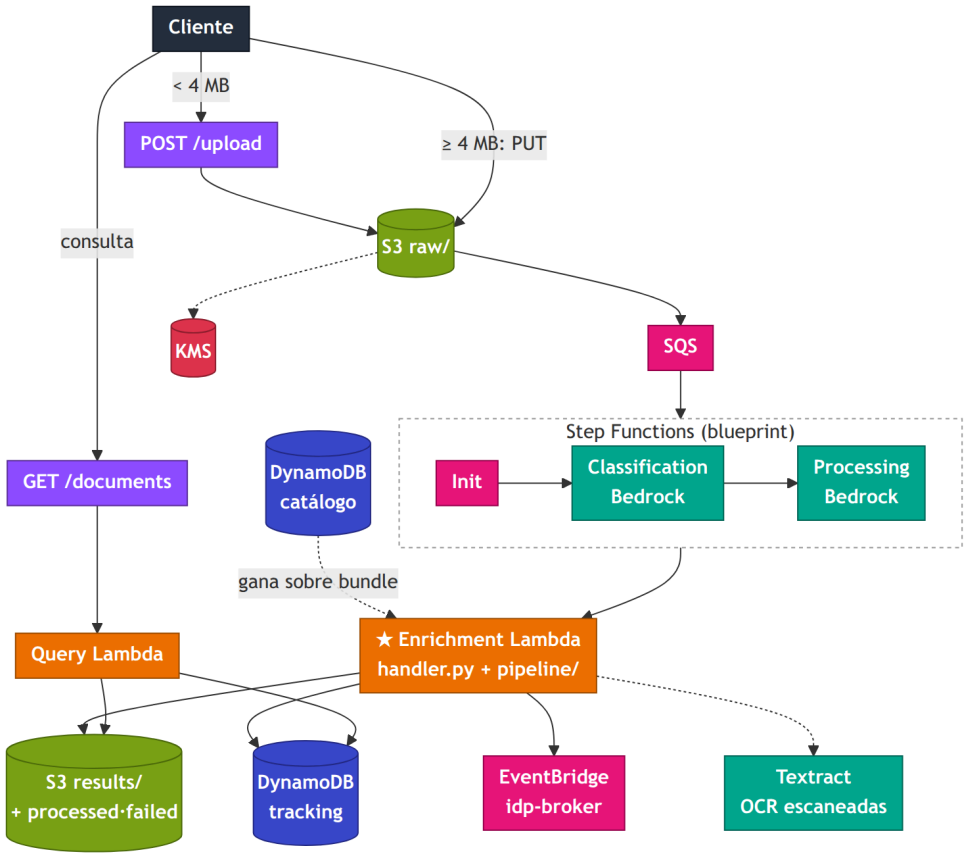


Diagrama de arquitectura AWS: Cliente sube un documento por API Gateway o directo a S3 raw/, dispara SQS y Step Functions (Classification/Processing con Bedrock), la Enrichment Lambda ejecuta el pipeline y escribe en S3 results/, DynamoDB tracking y EventBridge; el catálogo vive en DynamoDB; Query Lambda atiende las consultas.

Fuente editable: `diagrams/arquitectura-aws.mmd`.

Piezas clave:

Pieza	Rol
Upload Lambda	Valida el request, decide inline vs. presigned URL, escribe a `S3 raw/`. El prefijo `raw/{TIPO}/` es cómo se "fuerza" la clasificación.
S3 `raw/`	Recibir un objeto aquí dispara todo el pipeline (trigger vía SQS → Step Functions).

Pieza	Rol
Step Functions	Orquestador del blueprint. Para tipos aduanales, `ClassificationStep` / `ProcessingStep` corren pero su resultado se **descarta** — el handler re-clasifica y re-extrae dentro del pipeline de expediente (ver "Oportunidades de mejora" en `FLUJO_EJECUCION.md`).
Enrichment Lambda ⭐	El cerebro: aquí vive `handler.py` y todo `pipeline/`. Ver diagrama de flujo abajo.
Textract	OCR **solo** en páginas de PDF sin capa de texto (el resto usa pypdf, gratis).
DynamoDB catálogo	Tipos y reglas editables en caliente (`pnpm sync`), `RETAIN` — sobrevive a un `destroy` del stack. Gana siempre sobre el JSON empaquetado en runtime.
DynamoDB tracking	Envelope **compacto** por documento (escalares, `arrayCounts`, `liquidacionNormalizada`, `tracking`) — acotado por el límite de payload de Step Functions (256 KB).
S3 `results/`	Resultado **completo**: todos los campos, arrays completos, checks y confianzas.
EventBridge (`idp-broker`)	Publica el resultado final (`document-processed`) para que sistemas externos se suscriban.
Query Lambda	`GET /documents/{id}` y `GET /documents?docType=&field=` — lee DynamoDB (indexado por GSI `DocumentTypeIndex` cuando hay `docType`, `Scan` si no) y S3 (`fullResult` con `full=1`).
KMS	Cifra el bucket S3. Es el rubro de infraestructura (no-IA) más caro medido — ver `COSTOS.md`.

Seguridad por ambiente (API keys, WAFv2, dominio propio) solo aplica a **prod**, gateada por `environment === "prod"` — detalle completo en `AMBIENTES_Y_SEGURIDAD.md`.

Flujo de ejecución

El `handler.py` de la Enrichment Lambda bifurca en dos caminos según el tipo de documento clasificado. Mismos colores por servicio que el diagrama de arquitectura: 🟠 Compute (Lambda/lógica propia) · 🟢 Storage (S3) · 🔵 Database (DynamoDB) · 🩷 App Integration (EventBridge/Step Functions) · 🩵 Machine Learning (Bedrock/Textract).

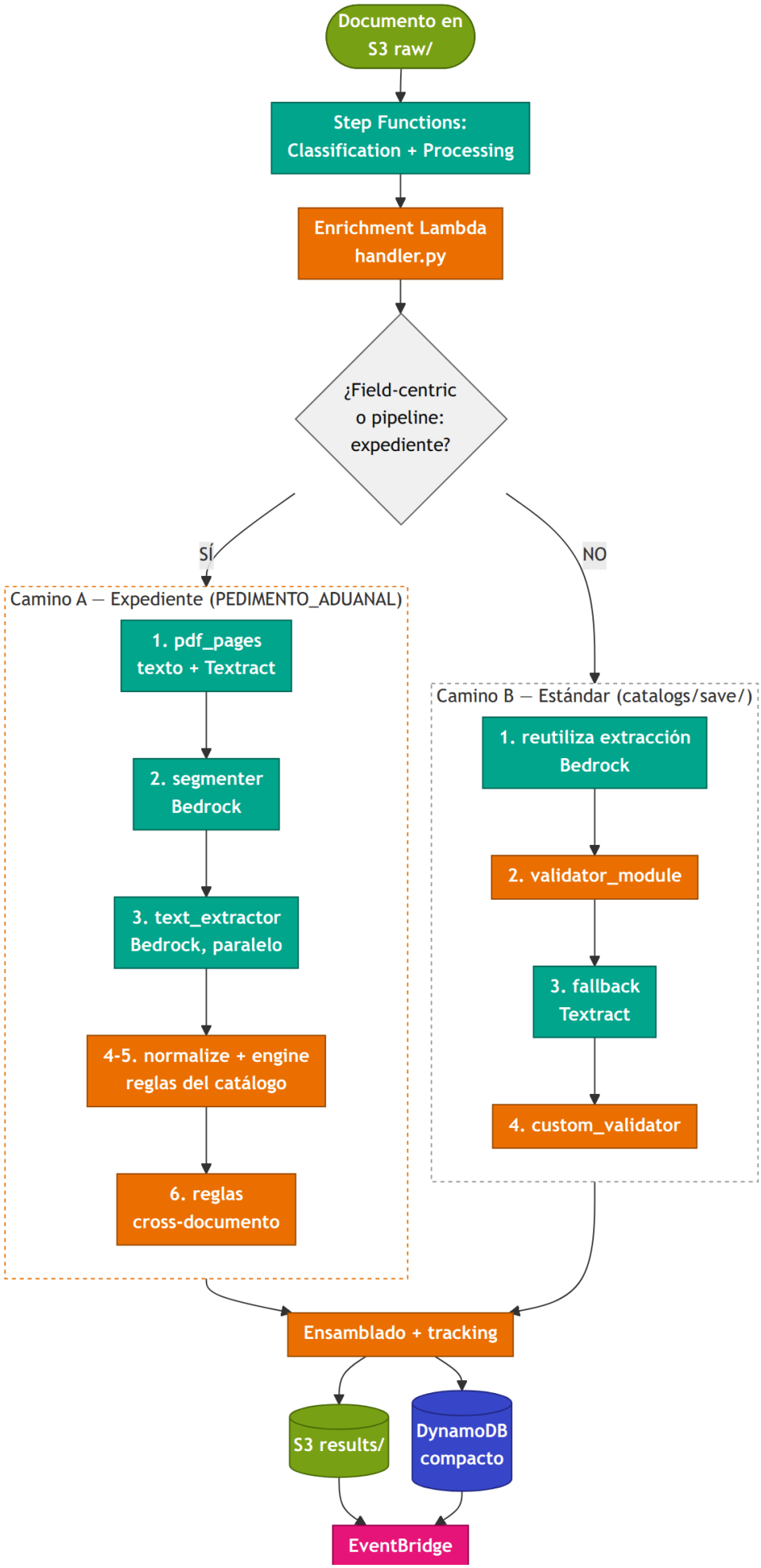

Documento en S3 raw/
Step Functions: Classification + Processing
Enrichment Lambda handler.py
¿Field-centric o pipeline: expediente?
SÍ
NO
Camino A — Expediente (PEDIMENTO_ADUANAL)
1. pdf_pages texto + Textract
2. segmenter Bedrock
3. text_extractor Bedrock, paralelo
4-5. normalize + engine reglas del catálogo
6. reglas cross-documento
Camino B — Estándar (catalogs/save/)
1. reutiliza extracción Bedrock
2. validator_module
3. fallback Textract
4. custom_validator
Ensamblado + tracking
S3 results/
DynamoDB compacto
EventBridge

Diagrama de flujo de ejecución: un documento en S3 raw/ pasa por Step Functions (Bedrock), la Enrichment Lambda decide entre el Camino A (pipeline de expediente: pdf_pages, segmenter, text_extractor, normalize+engine, reglas cross-documento) o el Camino B (flujo estándar: reutiliza extracción, validator_module, fallback Textract, custom_validator); ambos terminan en Ensamblado, se persisten en S3 results/ y DynamoDB, y publican en EventBridge.

Fuente editable: `diagrams/flujo-ejecucion.mmd`.

Notas de enrutamiento y detalle por paso (recortado del diagrama para que quepa en una vista — el detalle completo está aquí y en `FLUJO_EJECUCION.md`):

- La condición de la izquierda decide el **camino de extracción**; `pipeline: "expediente"` en el catálogo controla además, por separado, la **candidatura a auto-segmentación** (solo tipos con ese flag se consideran cuando el PDF se sube sin declarar tipo) y la **elegibilidad para reglas cross-documento**. Detalle: `../catalogs/README.md`.
- **Camino A, paso 1**: `pdf_pages.load_pages` — texto nativo (pypdf, layout) + Textract OCR **en paralelo** solo en páginas escaneadas.
- **Camino A, paso 2**: `segmenter.segment_pages` — si el uploader declaró el tipo (`declared_type`), 1 solo segmento sin llamar al modelo; si no, Bedrock clasifica los rangos de página.
- **Camino A, paso 3**: extracción por segmento **paralela** (`ThreadPoolExecutor`, env var `EXTRACT_WORKERS`) vía `text_extractor.extract_document`, con el schema del catálogo.
- **Camino A, paso 4-5**: `normalize._recover_references` (regex) + `liquidacion.build_liquidacion_normalizada` + `qr_recovery` (si `QR_RECOVERY_ENABLED`), luego `engine.run_rules` / `resolve_fields` / `document_status`.
- **Camino A, paso 6**: `_run_cross_document_rules` — mismo pedimento, RFC importador, UUIDs únicos, conciliación de gastos/valores.
- El Camino B (`custom_validators/*.mjs`, `validator_module`) **no corre** para tipos field-centric como `PEDIMENTO_ADUANAL` — solo aplica a los tipos legacy aparcados en `catalogs/save/`.
- Tiempos medidos: 1 página ≈ 30 s · 27 páginas escaneadas ≈ 4 min · 86 páginas ≈ 6-8 min (límite duro de la Lambda: 15 min).

Recorrido con referencias exactas a archivo:línea y la lista priorizada de mejoras de dataflow pendientes: **`FLUJO_EJECUCION.md`**.